

EcoFlow — Federated Supply Chain Decarbonization & CBAM Auditor

Complete Competition Blueprint: Kaggle × Google Gen AI Agent Capstone

This document is the single source of truth for building a competition-winning EcoFlow submission.

Version 1.0 | June 2026 | Target: Kaggle × Google 5-Day Gen AI Agent Championship

Table of Contents

1. [Executive Summary](#)
2. [Winning Strategy Analysis](#)
3. [Architecture Overview](#)
4. [Technology Selection Matrix](#)
5. [Agent System Design](#)
6. [A2A Implementation](#)
7. [FastMCP Design](#)
8. [Data Strategy](#)
9. [Repository Structure](#)
10. [Database Design](#)
11. [API Design](#)
12. [Frontend Design](#)
13. [AI-Assisted Development Workflow](#)
14. [Day-by-Day Execution Plan](#)
15. [Testing Strategy](#)
16. [Demo Strategy](#)
17. [Kaggle Submission Strategy](#)
18. [Documentation Plan](#)
19. [Risk Analysis](#)
20. [Winning Enhancements](#)
21. [Final Winning Roadmap](#)

1. Executive Summary

Problem

Global supply chains are the single largest contributor to corporate carbon emissions, accounting for up to 70% of a company's total carbon footprint (Scope 3 emissions). As of January 2026, the EU Carbon Border Adjustment Mechanism (CBAM) is fully operational, requiring importers to declare and pay for the carbon content of goods crossing EU borders. The SEC's climate disclosure rules mandate Scope 3 reporting for large US public companies.

Non-compliance penalties reach €50 per tonne of CO_2 — and companies simply do not have the tooling to calculate, verify, or report this at scale. Manual Scope 3 auditing currently takes 3–6 months per supplier, requires specialist consultants billing \$300/hour, and produces reports that are immediately out of date.

The problem is not data availability — EPA emission factors, Eurostat industrial carbon coefficients, and IEA energy intensity factors are all publicly available. The problem is orchestration: no system can ingest a company's logistics records, automatically query supplier-side emissions data, cross-check against regulatory databases, and produce a certified audit report in minutes rather than months.

Opportunity

EcoFlow solves this with a **federated multi-agent AI system** where a central orchestration hub communicates with autonomous supplier agents via the **Agent-to-Agent (A2A) protocol**, calculates Scope 3 emissions using **FastMCP-exposed carbon databases**, audits compliance against CBAM and SEC rules, and generates a certified, verified carbon audit report — entirely autonomously. This is not a demo of AI. This is a demonstration of **the future of enterprise regulatory compliance**.

Why This Project Can Win

1. **Stack alignment is maximal.** EcoFlow uses ADK 2.0, A2A protocol, FastMCP, Gemini 2.5 Pro/Flash, Cloud Run, Vertex AI, and Antigravity 2.0 — every major technology introduced in the course.
2. **The A2A implementation is genuine.** Most submissions will mock agent communication. EcoFlow runs real A2A message exchanges between independently deployed agents on Cloud Run.
3. **The problem is topical and verifiable.** CBAM compliance is a real 2026 legal requirement. Judges can verify the regulatory accuracy of outputs.
4. **The output is concrete.** A carbon audit PDF with Scope 3 calculations, compliance scores, and supplier-level breakdowns — not a chatbot response.
5. **The domain is uncrowded.** Unlike wildfire/pandemic/cybersecurity, supply chain decarbonization is not the first thing competition participants think of.

Key Differentiators

- Only submission using genuine A2A federation across multiple independently deployed agents
- Real CBAM compliance scoring with publicly auditable regulatory references
- Federated architecture where supplier agents are first-class participants, not data sources
- Carbon audit output that could be presented to a real compliance officer the next day
- Antigravity 2.0 visual task monitoring showing every agent decision in real time

2. Winning Strategy Analysis

Why EcoFlow Was Selected

The LLM Council analysis (run prior to this blueprint) ranked EcoFlow #1 of 6 competing ideas with an overall score of 87/100 based on five independent expert perspectives. The decisive factors:

- **A2A is the crown jewel of the course stack.** The competition is built around Google ADK 2.0, and A2A (Agent-to-Agent) protocol is the most advanced, newest, and least-understood capability in that stack. A submission that correctly implements A2A federation between independently deployed agents demonstrates the highest level of course mastery. No other idea in the candidate pool used A2A as a core architectural primitive.
- **Regulatory compliance is inherently verifiable.** Unlikely wildfire simulations or epidemic models where judges must trust the creator's claims about accuracy, CBAM regulations are public documents. Judges can verify that EcoFlow's compliance audit correctly references Article 7 of EU Regulation 2023/956. This credibility is rare in competition submissions.
- **The federated design is architecturally honest.** When EcoFlow simulates supplier agents, it does so by deploying them as independent Cloud Run services that communicate via A2A. The "simulation" is the architecture — not a workaround.

What Judges Care About

Based on the course's stated evaluation criteria and the psychology of technical competition judging:

Judges deeply care about:

- Evidence that the builder learned the course material (ADK 2.0, A2A, MCP usage)
- Agent behavior that is genuinely autonomous, not scripted chatbot responses
- Real data — even if limited — not fully synthetic outputs
- A demo they can follow without a 10-minute explanation
- Code quality visible in the notebook: structure, comments, agent interaction logs
- Deployment artifacts: a live URL signals professional execution
- A clear problem → agent action → verifiable output loop

Judges care somewhat about:

- Domain importance (EcoFlow scores high here — CBAM is genuinely pressing)
- Novelty of the application
- UI polish (matters less than technical depth, but a bad UI hurts)

Judges largely ignore:

- The size of the training dataset
- Whether the problem has been "solved before"
- Marketing language in the writeup
- Features that aren't demonstrated in the demo

What Typical Teams Will Build

Surveying the competition landscape, the average submission will be:

- A single Gemini agent with tool calling, wrapped in a Streamlit UI
- A CrewAI or LangChain pipeline described as "multi-agent" but running sequentially

- A chatbot solving a relatable problem (recipe suggestions, travel planning, customer support)
- No deployment beyond a Kaggle notebook
- No A2A, no real MCP servers, no ADK 2.0 workflows

The median submission will use Google GenAI SDK for simple tool calling and call it "agentic." This is the bar to clear.

How to Stand Out

- **Level 1 (clear 80% of submissions):** Use ADK 2.0 with proper multi-agent orchestration. Not LangChain. Not plain Gemini API calls. ADK 2.0 with defined agent roles, tool registration, and session management.
- **Level 2 (clear 95% of submissions):** Deploy real MCP servers. Not local mock functions registered as tools — actual FastMCP servers with proper schemas running independently.
- **Level 3 (clear 99% of submissions):** Implement genuine A2A federation. Two or more independently deployed Cloud Run services communicating via the A2A protocol with proper agent cards at /.well-known/agent.json.
- **Level 4 (win):** Do all of the above, with a real regulatory compliance problem, a deployable output (PDF audit report), a live demo URL, and a writeup that explains every architectural decision in terms of the course's own vocabulary.

What Would Make Judges Discuss EcoFlow After Judging

"That one with the actual federated agents — where the supplier agents were running on Cloud Run and the hub was making A2A calls to them in real time? That was the only submission that actually used the protocol the way it was designed. And the CBAM compliance report at the end — that was something you could actually hand to a lawyer."

How to Maximize Award Probability

1. **Make the A2A communication visible.** The Antigravity 2.0 dashboard showing live A2A message exchanges is the single most impactful visual. Set this up before anything else.
2. **Use real numbers.** EPA Emission Factors for Greenhouse Gas Inventories (2024 edition) gives actual kg \$CO₂e\$ per unit for hundreds of materials. Use them. Judges will recognize real emission factors.
3. **Generate a real PDF output.** A carbon audit report that looks like it came from a Big Four consulting firm — complete with Scope 3 breakdown by supplier, CBAM calculation, and compliance score — is the demo's money shot.
4. **Deploy to Cloud Run before Day 4.** A live URL in the Kaggle writeup is a quality signal judges associate with professional execution.
5. **Write the Kaggle notebook as if teaching the course.** Judges are also evaluating whether the submission demonstrates understanding of the material.

3. Architecture Overview

System Architecture

Code snippet

```
graph TD
    subgraph "User Interface Layer"
        UI[Streamlit Dashboard]
        NB[Kaggle Notebook]
    end

    subgraph "EcoFlow Hub — Cloud Run"
        ORCH[Orchestrator Agent<br/>ADK 2.0 LlmAgent]
        DOC[Document Extractor Agent<br/>ADK 2.0 LlmAgent]
        CARB[Carbon Engine Agent<br/>ADK 2.0 LlmAgent]
        COMP[Compliance Auditor Agent<br/>ADK 2.0 LlmAgent]
        LOG[Logistics Optimizer Agent<br/>ADK 2.0 LlmAgent]
        SESSION[Vertex AI Session Service]
        MEMORY[Agent Memory Store]
    end

    subgraph "FastMCP Servers — Cloud Run"
        MCP1[carbon-db-mcp<br/>Emission Factors DB]
        MCP2[cbam-reg-mcp<br/>CBAM Regulatory DB]
        MCP3[logistics-mcp<br/>Shipping Emissions DB]
    end

    subgraph "Supplier Agent Network — Cloud Run"
        SUP1[Supplier Agent: SteelCo<br/>A2A Server]
        SUP2[Supplier Agent: FreightHub<br/>A2A Server]
        SUP3[Supplier Agent: PackagingLtd<br/>A2A Server]
    end

    subgraph "Google Cloud Backend"
        VERTEX[Vertex AI<br/>Gemini 2.5 Pro]
        GCS[Cloud Storage<br/>Audit Reports]
        BQ[BigQuery<br/>Historical Audits]
        ANTGRAV[Antigravity 2.0<br/>Agent Monitoring]
    end
```



Agent Architecture

Code snippet

```
graph LR
    subgraph "ADK 2.0 Agent Graph"
        direction TB
        ORCH["Orchestrator<br/>Root Agent"]
        DOC["Document<br/>Extractor"]
        CARB["Carbon<br/>Engine"]
        COMP["Compliance<br/>Auditor"]
        LOG["Logistics<br/>Optimizer"]
    end

    ORCH -->|"1. Extract components"| DOC
    DOC -->|"2. Component manifest"| CARB
    CARB -->|"3. Emission totals"| COMP
    COMP -->|"4. Compliance gaps"| LOG
    LOG -->|"5. Audit report"| ORCH
```

```
COMP-->["Violation: recalibrate"] CARB
LOG-->["Data gap: A2A request"] DOQ
end
```

A2A Communication Architecture

Code snippet

```
sequenceDiagram
    participant HUB as EcoFlow Hub<br/>(Orchestrator)
    participant LOG as Logistics Optimizer<br/>(AIK Agent)
    participant A2A as A2A Protocol Layer
    participant SUP as SteelCo Supplier Agent<br/>(Cloud Run)

    HUB->>LOG: Task: resolve supplier emissions gap
    LOG->>A2A: Discover agent at steelco.ecoflow.api
    A2A->>SUP: GET /well-known/agent.json
    SUP-->>A2A: AgentCard (capabilities, schema)
    A2A-->>LOG: Agent discovered
    LOG->>A2A: POST /a2a/tasks/send
    Note over LOG,SUP: Task: {material: "hot-rolled steel",<br/>quantity: 450t, period: 2026-Q1}
    A2A->>SUP: Forward task
    SUP-->>SUP: Compute emissions from<br/>local production data
    SUP-->>A2A: TaskResult {co2e_kg: 810000,<br/>certificate: "SteelCo-2026-Q1-042"}
    A2A-->>LOG: Result received
    LOG->>HUB: Supplier data resolved, updating audit
```

Data Flow Architecture

Code snippet

```
owchart TD
    INPUT["INPUT / 📄 Input: Logistics Records<br/>(PDF, CSV, JSON)"]
```



Infrastructure Architecture

Code snippet

```
graph TD
    subgraph "Cloud Run Services"
        HUB["ecoflow-hub<br/>8080<br/>Main orchestration service"]
        MCP_CARBON["carbon-db-mcp<br/>8081<br/>FastMCP server"]
        MCP_CBAM["cbam-reg-mcp<br/>8082<br/>FastMCP server"]
        MCP_LOGISTICS["logistics-mcp<br/>8083<br/>FastMCP server"]
        SUP_STEEL["supplier-steelco<br/>8084<br/>A2A server"]
        SUP_FREIGHT["supplier-freighthub<br/>8085<br/>A2A server"]
        SUP_PACKAGING["supplier-packaging<br/>8086<br/>A2A server"]
    end
```

```
    subgraph "Vertex AI"
        GEMINI_PRO["gemini-2.5-pro<br/>Deep reasoning agents"]
        GEMINI_FLASH["gemini-2.5-flash<br/>High-throughput parsing"]
    end
```

```
    subgraph "Supporting Services"
        AGMON["Antigravity 2.0<br/>Agent monitoring"]
        VSESS["Vertex AI Session Service<br/>State management"]
        GCS["Cloud Storage<br/>Report artifacts"]
        BIGQ["BigQuery<br/>Audit history"]
    end
```

```
    HUB --> GEMINI_PRO
    HUB --> VSESS
    HUB --> AGMON
    HUB <--> MCP_CARBON
    HUB <--> MCP_CBAM
    HUB <--> MCP_LOGISTICS
    HUB --> A2A[SUP_STEEL]
    HUB --> A2A[SUP_FREIGHT]
    HUB --> A2A[SUP_PACKAGING]
    MCP_CARBON --> GEMINI_FLASH
    SUP_STEEL --> GEMINI_FLASH
    HUB --> GCS
    HUB --> BIGQ
```

4. Technology Selection Matrix

Selected Technologies

Google ADK 2.0 (Agent Development Kit)

- **Why selected:** ADK 2.0 is the primary framework introduced in this course. Judges will be familiar with its patterns and actively look for its usage. Using plain LangChain or raw Gemini API calls signals that the builder did not engage with course content.
- **Judge impact:** High. Judges will inspect code and recognize ADK 2.0 patterns (LlmAgent, SequentialAgent, ParallelAgent, WorkflowAgent). Using these correctly is the difference between "I built an AI app" and "I mastered the course material."
- **Implementation:** Use `google.adk.agents.LlmAgent` for all reasoning agents. Use `google.adk.agents.SequentialAgent` for the main pipeline. Use `google.adk.agents.ParallelAgent` for concurrent supplier queries. Register all tools using `@tool` decorator.
- **Score impact:** +15 points vs. not using ADK 2.0
- **Risk:** Medium. ADK 2.0 has breaking changes from ADK 1.x. Use ADK 2.0 exclusively. Never import from `google.adk.v1`.

A2A Protocol (Agent-to-Agent)

- **Why selected:** A2A is the single most differentiating technology in the course stack. It enables true federation between independently deployed agents. No other project idea in the candidate pool uses A2A as its core architectural primitive.
- **Judge impact:** Maximum. Any judge who understands A2A will immediately recognize that a genuine A2A implementation is orders of magnitude more complex than simulated multi-agent communication.
- **Implementation:** Deploy supplier agents as independent Cloud Run services. Each exposes `GET /.well-known/agent.json` (AgentCard) and `POST /a2a/tasks/send`. Use the official `a2a-sdk` Python package from pypi.org/project/a2a-sdk.
- **Score impact:** +20 points vs. not using A2A
- **Risk:** Medium-High. A2A implementation requires understanding of the protocol spec. Mitigation: use the official SDK and follow the ADK quickstart guide at adk.dev/a2a/quickstart-exposing/.

FastMCP

- **Why selected:** MCP (Model Context Protocol) is how agents connect to external data sources and tools in a standardized way. FastMCP is the Python implementation. Using it correctly demonstrates understanding of agent-tool integration patterns.
- **Judge impact:** High. Judges will look for evidence of proper tool integration beyond `@tool` decorated functions. MCP servers are a professional pattern.
- **Implementation:** Three dedicated MCP servers: `carbon-db-mcp` (EPA emission factors), `cbam-reg-mcp` (EU regulatory database), `logistics-mcp` (shipping emission factors). Each

deployed independently on Cloud Run.

- **Score impact:** +10 points vs. using only registered tool functions
- **Risk:** Low. FastMCP is stable and well-documented at adk.dev/mcp/.

Gemini 2.5 Pro + Flash

- **Why selected:** Gemini 2.5 Pro for complex reasoning agents (Compliance Auditor, Orchestrator). Gemini 2.5 Flash for high-throughput parsing tasks (Document Extractor, Supplier Agents). Using both shows understanding of cost-performance tradeoffs.
- **Judge impact:** Medium. Using the latest Gemini models signals current knowledge. More importantly, using Pro for reasoning and Flash for parsing shows architectural maturity.
- **Implementation:** model="gemini-2.5-pro" for Orchestrator, Carbon Engine, Compliance Auditor. model="gemini-2.5-flash" for Document Extractor, Supplier Agents, Logistics Optimizer.
- **Score impact:** +5 points vs. using a single model for everything
- **Risk:** Low.

Cloud Run

- **Why selected:** Deployment is the most underrated quality signal in competition submissions. A live URL demonstrates that the system works outside a notebook. Cloud Run is the natural deployment target for ADK 2.0 agents.
- **Judge impact:** High. "Deployed and accessible at: <https://ecoflow-hub-xxx.run.app>" in the writeup immediately elevates the submission above 90% of competitors.
- **Implementation:** Containerize each service with Docker. Use gcloud run deploy with appropriate memory/CPU settings. Enable HTTPS by default.
- **Score impact:** +10 points vs. notebook-only submission
- **Risk:** Low. Cloud Run deployment is well-documented and free tier is sufficient.

Vertex AI Session Service

- **Why selected:** State management across agent turns. Using VertexAiSessionService instead of InMemorySessionService shows production-readiness.
- **Judge impact:** Medium. Shows understanding of stateful agent systems.
- **Implementation:** Replace InMemorySessionService with VertexAiSessionService in the orchestrator. Configure with proper GCP project and location.
- **Score impact:** +5 points
- **Risk:** Low.

Antigravity 2.0

- **Why selected:** Real-time visual monitoring of agent decisions is the most compelling demo element. Antigravity 2.0 shows every tool call, A2A message, and agent decision in a beautiful timeline interface.
- **Judge impact:** Maximum for demo quality. When judges see live agent activity in

Antigravity, they understand that the agents are genuinely running and making autonomous decisions.

- **Implementation:** Configure Antigravity 2.0 to connect to the EcoFlow Hub Cloud Run service. Show the A2A communication timeline during the demo.
- **Score impact:** +10 points (demo quality)
- **Risk:** Medium. Antigravity 2.0 is relatively new. Test locally first.

ADK 2.0 Dynamic Workflows

- **Why selected:** The compliance audit loop (audit → violation detected → recalibrate → re-audit) is a natural fit for dynamic workflow routing. Static sequential pipelines don't demonstrate agentic behavior.
- **Judge impact:** High. The self-correction loop is what distinguishes an agentic system from a pipeline.
- **Implementation:** Use ADK 2.0's @node decorator with conditional routing. Compliance score below threshold → route back to Carbon Engine with corrective context.
- **Score impact:** +8 points
- **Risk:** Medium.

Excluded Technologies

LangChain / LangGraph

- **Why excluded:** Judges are evaluating ADK 2.0 competency. Using LangChain signals that the builder defaulted to familiar tools rather than learning course material. Even if LangGraph is more familiar, ADK 2.0 must be the primary framework.

CrewAI

- **Why excluded:** Same reason as LangChain. Additionally, CrewAI's sequential agent pattern is simpler than ADK 2.0's workflow graphs and would score lower on technical depth.

OpenAI API

- **Why excluded:** This is a Google competition. Using non-Google models is actively counterproductive for scoring.

Kubernetes

- **Why excluded:** Over-engineering for a 5-day competition. Cloud Run provides sufficient orchestration with much lower operational overhead.

Kafka / Event Streaming

- **Why excluded:** Adds architectural complexity without sufficient scoring benefit. ADK 2.0's event system is sufficient for the competition timeline.

5. Agent System Design

5.1 Orchestrator Agent

Purpose

The root agent that receives user input, manages the overall audit workflow, maintains session state, and produces the final carbon audit report. All other agents are sub-agents of the Orchestrator.

Responsibilities

- Receive logistics document (PDF/CSV/JSON) from user
- Initialize audit session with VertexAiSessionService
- Coordinate the document extraction → carbon calculation → compliance audit → logistics optimization pipeline
- Handle errors and retries from sub-agents
- Generate and store the final PDF audit report
- Return structured summary to user

Inputs

JSON

```
{  
  "company_name" "Acme Manufacturing GmbH"  
  "reporting_period" "2026-Q1"  
  "logistics_document" "<base64_encoded_pdf_or_csv_path>"  
  "target_market" "EU"  
  "cbam_applicable" true  
}
```

Outputs

JSON

```
{  
  "audit_id": "ECO-2026-Q1-00042",  
  "company": "Acme Manufacturing GmbH",  
  "total_scope3_co2e_tonnes": 1247.3,  
  "cbam_compliance_score": 0.847,  
  "cbam_estimated_levy_eur": 49892,  
  "supplier_risk_matrix": ,  
  "remediation_roadmap": ,  
  "report_url": "https://storage.googleapis.com/ecoflow-reports/ECO-2026-Q1-00042.pdf",  
  "status": "COMPLIANT_WITH_WARNINGS"  
}
```

Memory

- **Session memory:** Current audit state (VertexAiSessionService)
- **Long-term memory:** Historical audit results for trend analysis (BigQuery)
- **Tool memory:** Cached emission factor lookups (in-session cache)

Tools

Python

@tool

def initialize_audit_session(company_name: str, reporting_period: str) -> dict:

"""Create a new audit session and return session ID."""

pass

@tool

def store_audit_result(session_id: str, result: AuditResult) -> str:

"""Persist audit result to BigQuery and Cloud Storage."""

pass

@tool

def generate_audit_report(session_id: str, audit_data: dict) -> str:

"""Generate PDF audit report using ReportLab and upload to GCS."""

pass

@tool

```
def get_historical_audits(company_name: str, limit: int = 5) -> list:
    """Retrieve previous audit results from BigQuery for trend analysis."""
    pass
```

System Prompt

Plaintext

You are EcoFlow's Orchestrator Agent, the chief carbon audit coordinator for a Fortune 500 supply chain decarbonization platform. Your role is to coordinate a team of specialized agents to produce a complete, accurate, and legally defensible carbon audit report compliant with EU CBAM Regulation 2023/956 and SEC climate disclosure requirements.

WORKFLOW

1. Receive the logistics document and company context
2. Delegate document extraction to the Document Extractor Agent
3. Pass the component manifest to the Carbon Engine Agent for emission calculations
4. Route emission totals to the Compliance Auditor Agent
5. If compliance gaps exist, coordinate with the Logistics Optimizer Agent for remediation
6. Generate the final audit report
7. Return a structured summary to the user

CRITICAL REQUIREMENTS

- Every carbon figure must cite its source emission factor (EPA, IEA, or IPCC AR6)
- Every compliance assessment must reference the specific regulatory article
- If a supplier's data is missing, initiate A2A data request before proceeding
- Never estimate CBAM levies without first verifying current carbon prices from the regulatory database

OUTPUT QUALITY STANDARD

Your report must be suitable for submission to EU customs authorities. Treat every output as if a compliance officer will present it to regulators.

Failure Recovery

- Sub-agent timeout → retry with exponential backoff (max 3 attempts)
- Missing supplier data → initiate A2A request, flag as "estimated" if no response within 30s

- PDF generation failure → return JSON report as fallback
- Session loss → reconstruct from VertexAiSessionService checkpoint

5.2 Document Extractor Agent

Purpose

Parse raw logistics documents (shipping manifests, bills of lading, invoices, customs declarations) and extract structured component data suitable for carbon calculation.

Responsibilities

- Accept multiple document formats: PDF, CSV, JSON, XML, XLSX
- Use Gemini 2.5 Flash's multimodal capabilities to parse PDFs directly
- Extract: supplier names, material types, quantities (with units), transport legs, transport modes, origin/destination countries
- Normalize units (convert all quantities to metric tonnes, distances to km)
- Flag ambiguous entries for human review
- Output validated, structured component manifest

Inputs

- Raw document bytes (base64 encoded) or file path
- Document type hint (optional)
- Company context (for supplier name disambiguation)

Outputs

```
JSON
{
  "manifest_id" "MAN-20260315-001"
  "extraction_confidence" 0.94
  "components"
  {
    "line_item" 1
    "supplier_name" "SteelCo AG"
    "supplier_country" "DE"
    "material_type" "hot_rolled_steel"
    "quantity_tonnes" 450.0
    "transport_mode" "rail"
    "transport_distance_km" 340
  }
}
```



```

"origin_port" "Hamburg"
"destination_port" "Rotterdam"
"hs_code" "7208.51"
"extraction_confidence" 0.97
"needs_review" false
|
|
|
"flagged_items" |
"missing_fields" "transport_distance_km for line items 3, 7"
|

```

Tools

Python

@tool

def parse_pdf_document(pdf_base64: str) -> dict:

"""Use Gemini Vision to extract structured data from PDF shipping manifest."""

pass

@tool

def parse_csv_manifest(csv_content: str) -> dict:

"""Parse CSV logistics manifest into structured component data."""

pass

@tool

def normalize_units(quantity: float, unit: str, target_unit: str = "tonnes") -> float:

"""Convert quantity units to standard metric tonnes."""

pass

@tool

def lookup_hs_code(material_description: str) -> str:

"""Map material description to EU HS (Harmonized System) commodity code."""

pass

@tool

def validate_supplier_name(name: str, country: str) -> dict:

"""Validate supplier name against known supplier registry and return canonical name."""

pass

System Prompt

Plaintext

You are EcoFlow's Document Extractor Agent, a specialist in parsing international trade and logistics documents. Your task is to extract precise, structured supply chain data from logistics documents including:

- Bill of Lading (B/L)
- Commercial Invoice
- Packing List
- Customs Entry Declaration (SAD)
- CIMR Consignment Note
- Air Waybill (AWB)
- CSV logistics export

EXTRACTION RULES

1. Extract ALL line items — never summarize or aggregate without flagging it
2. Always convert quantities to metric tonnes (t) and distances to kilometres (km)
3. Map transport modes to: road | rail | sea | air | inland_waterway
4. If a supplier country is not explicit, infer from port of loading
5. Assign EU HS codes where material descriptions are clear
6. Flag any line item where confidence is below 0.85

CRITICAL: If a document contains a date, extract the reporting period. If absent, use the session context. If a field is genuinely missing, set it to null and add to missing_fields — never fabricate values.

OUTPUT: Return valid JSON matching the ComponentManifest schema exactly.

Prompt Template for PDF Parsing Tool

Python

```
"""ARISE PROMPT"""
```

Analyze this shipping/logistics document and extract all supply chain data. Return a JSON object with this EXACT structure:

```
{
  "document_type": "<bill_of_lading|invoice|customs_declaration|other>",
  "document_date": "<YYYY-MM-DD>",
  "shipper": "<company name>",
  "consignee": "<company name>",
  "line_items": [
    {
      "description": "<material description>",
      "quantity": <number>,
      "unit": "<tonnes|kg|units|m3>",
      "supplier_name": "<company name or null>",
      "supplier_country": "<ISO 3166-1 alpha-2 or null>",
      "transport_mode": "<road|rail|sea|air|inland_waterway|null>",
      "origin": "<city or port name or null>",
      "destination": "<city or port name or null>",
      "distance_km": <number or null>,
      "hs_code": "<6-digit HS code or null>"
    }
  ]
}
```

IMPORTANT:

- Use null for missing fields, never "" or 0
- Quantities must be positive numbers
- Country codes must be 2-letter ISO codes
- Return ONLY the JSON object, no preamble

```
"""
```

5.3 Carbon Engine Agent

Purpose

Calculate Scope 3 greenhouse gas emissions for each supply chain component using internationally recognized emission factors, accessed via the carbon-db-mcp FastMCP server.

Responsibilities

- Accept component manifest from Document Extractor
- Query carbon-db-mcp for material-specific emission factors (Scope 3 Category 1: purchased goods)
- Query logistics-mcp for transport-mode emission factors (Scope 3 Category 4: upstream transport)
- Apply GHG Protocol calculation methodology (AR6 GWP100 values)
- Aggregate emissions by supplier, material category, and transport leg

- Flag suppliers with missing emission data for A2A resolution
- Return verified emission totals with full calculation audit trail

Inputs

- Component manifest (from Document Extractor)
- Calculation methodology preference (default: GHG Protocol Corporate Standard)

Outputs

JSON

```
{
  "calculation_id": "CALC-20260315-001",
  "methodology": "GHG Protocol Corporate Value Chain Standard, AR6 GWP100",
  "reporting_period": "2026-Q1",
  "total_scope3_co2e_tonnes": 1247.3,
  "by_supplier": {
    "supplier_name": "SteelCo AG",
    "supplier_country": "DE",
    "material_co2e_tonnes": 810.0,
    "transport_co2e_tonnes": 12.4,
    "total_co2e_tonnes": 822.4,
    "emission_factor_source": "EPA 2024 Emission Factors - Table 2.1",
    "emission_factor_kg_co2e_per_tonne": 1800,
    "data_quality": "verified",
    "calculation": "450t × 1.8 tCO2e/t = 810 tCO2e"
  },
  "by_category": {
    "scope3_cat1_purchased_goods": 1178.5,
    "scope3_cat4_upstream_transport": 68.8
  },
  "missing_data_suppliers": "PackagingLtd",
  "a2a_requests_needed": "PackagingLtd"
}
```

Tools

Python

@tool

```
def get_material_emission_factor(material_type: str, production_country: str, year: int = 2024) ->
```

```
dict:
```

```
"""
```

```
    Query carbon-db-mcp for material emission factor.
```

```
    Returns: {factor kg_co2e_per_tonne, source, methodology, confidence}
```

```
"""
```

```
    pass
```

@tool

```
def get_transport_emission_factor(transport_mode: str, fuel_type: str = "diesel", load_factor: float = 0.7) -> dict:
```

```
"""
```

```
    Query logistics-mcp for transport emission factor.
```

```
    Returns: {factor kg_co2e_per_tonne_km, source, vehicle_type}
```

```
"""
```

```
    pass
```

@tool

```
def calculate_scope3_category1(material_type: str, quantity_tonnes: float, emission_factor: float) ->
```

```
dict:
```

```
    """Calculate Scope 3 Category 1 emissions for purchased goods."""
```

```
    pass
```

@tool

```
def calculate_scope3_category4(quantity_tonnes: float, distance_km: float, transport_mode: str, emission_factor: float) -> dict:
```

```
    """Calculate Scope 3 Category 4 emissions for upstream transport."""
```

```
    pass
```

@tool

```
def aggregate_by_supplier(calculations: list) -> dict:
```

```
    """Aggregate emission calculations by supplier with verification."""
```

```
    pass
```

System Prompt

Plaintext

You are EcoFlow's Carbon Engine Agent, an expert in greenhouse gas accounting under the GHG Protocol Corporate Value Chain (Scope 3) Standard. Your role is to calculate precise, auditable Scope 3 emissions for supply chain components using internationally recognized emission factors.

CALCULATION METHODOLOGY

Primary standard: GHG Protocol Corporate Value Chain Standard (2011, updated 2024)
GWP values: IPCC AR6, 100-year Global Warming Potential
Scope 3 Category 1 (Purchased Goods): material production emissions
Scope 3 Category 4 (Upstream Transportation): transport emissions
Unit: metric tonnes of CO₂ equivalent (tCO₂e)

EMISSION FACTOR HIERARCHY (use in order)

1. Supplier-specific factors (from A2A verified data) — highest quality
2. Industry-average factors from carbon-db-mcp (EPA, IEA, Ecoinvent)
3. Country-specific grid emission factors for energy-intensive materials
4. IPCC default factors — only as last resort, flag as "estimated"

CALCULATION RULES

1. Never use a factor without citing its source (database, table, year)
2. Show the full calculation: quantity × factor = result
3. If emission factor confidence < 0.7, flag as "low confidence estimate"
4. For missing suppliers, add to a2a_requests_needed list — do NOT estimate

VERIFICATION

Cross-check totals against industry benchmarks. If a result is >3x the industry average for that material category, flag for review before proceeding.

5.4 Compliance Auditor Agent

Purpose

Evaluate calculated emissions against CBAM regulations (EU Regulation 2023/956) and SEC climate disclosure rules, produce a compliance score, estimate levy obligations, and identify specific regulatory violations.

Responsibilities

- Query cbam-reg-mcp for current CBAM commodity thresholds and carbon prices
- Calculate CBAM levy liability per EU regulation methodology
- Score compliance against SEC Regulation S-K Item 1500 (climate disclosures)
- Identify specific regulatory articles being violated
- Generate remediation recommendations with priority ranking
- Implement self-correction loop: if recalibration is requested, re-evaluate with updated emission data

Self-Correction Loop

Plaintext

If compliance_score < 0.70 AND recalibration_available

→ Request Carbon Engine Agent to recalculate with supplier-specific factors

→ Re-run compliance assessment with updated totals

→ If score still < 0.70, flag for human review, do not block report

System Prompt

Plaintext

You are EcoFlow's Compliance Auditor Agent, an expert in EU carbon border regulation and SEC climate disclosure requirements.

PRIMARY REGULATORY FRAMEWORKS

1. EU Carbon Border Adjustment Mechanism (CBAM) — Regulation (EU) 2023/956

- In force: January 2026 (full implementation)

- Applies to: iron/steel, aluminium, cement, fertilisers, electricity, hydrogen

- Reporting: quarterly CBAM declarations via EU CBAM Registry

- Carbon price: EU ETS spot price (query cbam-reg-mcp for current price)

2. SEC Climate Disclosure Rule (Final Rule, March 2024)

- Scope 3 required for large accelerated filers when material

- Safe harbor: good faith estimates with disclosed methodology acceptable

- Timeline: fiscal year 2026 for large accelerated filers

COMPLIANCE SCORING (0.0 to 1.0)

- 1.0: Full compliance, all data supplier-specific, documentation complete

- 0.8-0.99: Compliant with minor gaps (estimated factors for <10% of emissions)

- 0.6-0.79: Conditional compliance (gaps requiring remediation within 90 days)

- Below 0.6: Non-compliant, immediate action required

CBAM LEVY CALCULATION

CBAM levy EUR = total embedded CO₂e tonnes

EU ETS carbon price EUR per tonne

Always cite: Article 7(1) of Regulation (EU) 2023/956 for embedded emissions

methodology

Always cite: Article 22 for CBAM certificate pricing methodology

5.5 Logistics Optimizer Agent

Purpose

When the Compliance Auditor identifies gaps or high-emission suppliers, the Logistics Optimizer initiates A2A communication with supplier agents to request verified emissions data, and recommends alternative logistics routes or supplier substitutions to reduce CBAM exposure.

Responsibilities

- Initiate A2A requests to supplier agents for missing or low-confidence emission data
- Parse and validate supplier agent responses
- Calculate potential CBAM savings from logistics optimization
- Rank remediation actions by ROI (CBAM savings / implementation cost)
- Generate actionable remediation roadmap with 30/60/90-day milestones

A2A Request Template

Python

A2A TASK TEMPLATE =

"id" "{uuid}"

"type" "emissions_verification_request"


```
"params"
  "supplier_name" "{supplier_name}"
  "material_type" "{material_type}"
  "quantity_tonnes" "{quantity}"
  "production_period" "{period}"
  "requested_data"
    "total_embedded_co2e_kg"
    "production_emission_factor_source"
    "energy_source_breakdown"
    "third_party_verification_certificate"
  "required_format" "GHG_Protocol_Scope3_Cat1"
  "deadline_seconds" 30
}
```

System Prompt

Plaintext

You are EcoFlow's Logistics Optimizer Agent, responsible for two tasks:

TASK 1 — SUPPLIER DATA FEDERATION

When emission data is missing or unverified, initiate A2A communication with supplier agents.

A2A Protocol

1. Discover supplier agent at: `https://{supplier_id}.ecoflow.app/well-known/agent.json`
2. Send emissions verification request via `POST /a2a/tasks/send`
3. Wait up to 30 seconds for response
4. If response received: validate format, extract emission data, pass to Carbon Engine
5. If no response: flag supplier as "unverified", use industry average with "estimated" tag

TASK 2 — LOGISTICS OPTIMIZATION

After receiving compliance audit results:

1. Rank suppliers by CBAM levy contribution (highest first)
2. For each high-risk supplier, calculate:
 - Potential CBAM savings from switching to lower-carbon alternative
 - Modal shift savings (e.g., road → rail, 70% emission reduction)

- Renewable energy premium required for supplier to achieve EU ETS parity
- 3. Generate remediation roadmap
 - 30 days: engage top 3 suppliers for emissions verification
 - 60 days: implement modal shift for transport legs > 500km
 - 90 days: renegotiate with non-compliant suppliers or qualify alternative

OUTPUT: Structured remediation plan with CBAM savings calculations for each action

6. A2A Implementation

Architecture

The A2A (Agent-to-Agent) protocol enables EcoFlow's supplier agents to operate as autonomous, independently deployed services that the hub can discover and communicate with programmatically. This is not simulation — each supplier agent runs as a real Cloud Run service.

Agent Card Schema

Each supplier agent exposes an agent card at GET /.well-known/agent.json:

JSON

```
{
  "id": "supplier-steelco-agent",
  "name": "SteelCo AG Emissions Agent",
  "version": "1.0.0",
  "description": "Autonomous emissions verification agent for SteelCo AG supply chain data. Provides verified Scope 3 Category 1 emission certificates for steel products.",
  "url": "https://supplier-steelco.ecoflow.app",
  "capabilities": {
    "streaming": false,
    "push_notifications": false,
    "state_transition_history": true
  },
  "skills": [
    {
      "id": "emissions_verification",
      "name": "Emissions Verification",
      "description": "Provide verified CO2e emission data for specified material quantities and"
    }
  ]
}
```

```
production periods"
  "input_modes" "application/json"
  "output_modes" "application/json"
}
{
  "authentication"
  "type" "bearer"
  "description" "Provide EcoFlow API key in Authorization header"
}
```

Message Schema

Task Request (Hub → Supplier Agent)

```
JSON
{
  "jsonrpc" "2.0"
  "method" "tasks/send"
  "id" "task-20260315-steelco-001"
  "params"
  {
    "id" "task-20260315-steelco-001"
    "message"
    {
      "role" "user"
      "parts"
      [
        {
          "type" "text"
          "text" "Please provide verified Scope 3 Category 1 emission data for the following order:\n-  
Material: hot-rolled steel (HS 7208.51)\n- Quantity: 450.0 metric tonnes\n- Production period:  
2026-Q1 (January 1 - March 31, 2026)\n- Reference: Purchase Order PO-2026-00134\n\nRequired  
data: total embedded CO2e in kg, emission factor methodology, and third-party verification  
certificate if available.\n\nResponse format: EmissionsVerificationResponse JSON schema v1.2"
        }
      ]
    }
  }
}
```

Task Response (Supplier Agent → Hub)

JSON

```
{
  "jsonrpc": "2.0",
  "id": "task-20260315-steelco-001",
  "result": {
    "id": "task-20260315-steelco-001",
    "status": "completed",
    "artifacts": [
      {
        "type": "emissions_certificate",
        "parts": [
          {
            "type": "data",
            "data": {
              "supplier_name": "SteelCo AG",
              "material_type": "hot_rolled_steel",
              "hs_code": "7208.51",
              "quantity_tonnes": 450.0,
              "production_period": "2026-Q1",
              "total_embedded_co2e_kg": 810000,
              "emission_factor_kg_per_tonne": 1800,
              "emission_factor_methodology": "ISO 14064-1:2018, World Steel Association 2024 factors",
              "energy_source": {
                "natural_gas": 0.45,
                "electricity_grid_DE": 0.35,
                "coal": 0.20
              },
              "third_party_verification": {
                "verifier": "Bureau Veritas",
                "certificate_id": "BV-SteelCo-2026Q1-004",
                "standard": "ISO 14064-3",
                "verification_date": "2026-04-15"
              }
            }
          }
        ]
      }
    ]
  }
}
```

```

[
  {
    "cbam_compliant": true,
    "response_timestamp": "2026-06-18T10:23:41Z"
  }
]

```

Implementation

Supplier Agent Server (FastAPI + A2A SDK)

```

Python
# supplier_agents/steelco/main.py
import json
from datetime import datetime
from fastapi import FastAPI, HTTPException, Depends
from fastapi.security import HTTPBearer
from a2a.server.apps import A2AStarletteApplication
from a2a.server.agent_execution import AgentExecutor
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.types import AgentCard, AgentSkill, AgentCapabilities

# Define the AgentCard
AGENT_CARD = AgentCard(
    id="supplier-steelco-agent",
    name="SteelCo AG Emissions Agent",
    version="1.0.0",
    description="Autonomous emissions verification agent for SteelCo AG",
    url="https://supplier-steelco.ecoflow.app",
    capabilities=AgentCapabilities(streaming=False),
    skills=[
        AgentSkill(
            id="emissions_verification"

```

```

        name="Emissions Verification"
        description="Provide verified CO2e emission data for steel products"
        input_modes="application/json"
        output_modes="application/json"
    )

```

```

class SteelCoExecutor(AgentExecutor):
    def __init__(self):
        import google.generativeai as genai
        self.model = genai.GenerativeModel("gemini-2.5-flash")

```

```

        # Simulated internal emissions database
        self.emissions_db = {
            "hot_rolled_steel": {"factor_kg_per_tonne": 1800, "methodology": "WSA 2024"},
            "cold_rolled_steel": {"factor_kg_per_tonne": 2100, "methodology": "WSA 2024"},
            "galvanized_steel": {"factor_kg_per_tonne": 2350, "methodology": "WSA 2024"}
        }

```

```

    async def execute(self, context, event_queue):
        task_text = context.message.parts[0].text

```

```

        # Use Gemini Flash to parse the request
        parse_prompt = f"""
        Extract from this emission verification request:
        - material_type (normalize to snake_case)
        - quantity_tonnes (float)
        - production_period (YYYY-QN format)

```

```

        Request: {task_text}

```

```

        Return JSON only:
        {{
            "material_type": "...",
            "quantity_tonnes": 0.0,
            "production_period": "..."
        }}
        """
        response = self.model.generate_content(parse_prompt)
        params = json.loads(response.text)

```

```

        material = params["material_type"]
        quantity = params["quantity_tonnes"]

```

```

        if material not in self.emissions_db:
            factor = 1900
            methodology = "Generic Steel Average - supplier-specific data unavailable"

```

```

else
    factor = self.emissions_db[material]["factor_kg_per_tonne"]
    methodology = self.emissions_db[material]["methodology"]

    total_co2e_kg = quantity * factor
    result = {
        "supplier_name": "SteelCo AG",
        "material_type": material,
        "quantity_tonnes": quantity,
        "production_period": params["production_period"],
        "total_embedded_co2e_kg": total_co2e_kg,
        "emission_factor_kg_per_tonne": factor,
        "emission_factor_methodology": methodology,
        "cbam_compliant": True,
        "response_timestamp": datetime.utcnow().isoformat() + "Z"
    }

# Complete the task
from a2a.types import new_agent_text_message
await event_queue.enqueue_event(
    new_agent_text_message(json.dumps(result))
)

# Build A2A application
from a2a.server.stores import InMemoryTaskStore
handler = DefaultRequestHandler
agent_executor = SteelCoExecutor()
task_store = InMemoryTaskStore()

app = A2AStarletteApplication
agent_card = AGENT_CARD
http_handler = handler
build()

```

Hub-Side A2A Client

Python

```

# ecoflow_hub/a2a_client.py
import httpx
import json
import asyncio
from a2a_client import A2AClient
from a2a.type import AgentCard, Message, TextPart, Task

class SupplierNetworkClient:
    """A2A client for communicating with supplier agents."""
    def __init__(self, api_key: str):
        self.api_key = api_key
        self.http_client = httpx.AsyncClient(
            headers={
                "Authorization": f"Bearer {api_key}"
            },
            timeout=35.0
        )
        self.discovered_agents: dict[str, AgentCard] = {}

    async def discover_supplier(self, supplier_url: str) -> AgentCard | None:
        """Discover supplier agent capabilities via AgentCard."""
        try:
            agent = await A2AClient.get_client_from_agent_card_url(
                self.http_client,
                f"{supplier_url}/.well-known/agent.json"
            )
            card = await agent.get_agent_card()
            self.discovered_agents[supplier_url] = card
            return card
        except Exception as e:
            print(f"A2A discovery failed for {supplier_url}: {e}")
            return None

    async def request_emissions_data(
        self, supplier_url: str, material_type: str, quantity_tonnes: float, production_period: str
    ) -> dict | None:
        """Send A2A task to supplier agent requesting emission verification."""
        try:
            agent = await A2AClient.get_client_from_agent_card_url(
                self.http_client,
                f"{supplier_url}/.well-known/agent.json"
            )
            request_text =
                f"Please provide verified Scope 3 Category 1 emission data:\n"
                f"- Material: {material_type}\n"

```



```

        f"- Quantity: {quantity_tonnes} metric tonnes\n"
        f"- Production period: {production_period}\n"
        f"Return as EmissionsVerificationResponse JSON."

```

```

    task: Task = await agent.send_message(
        message=Message(
            role="user",
            parts=[TextPart(text=request.text)]
        )
    )

```

```

    # Poll for completion (max 30s)

```

```

    for i in range(30):

```

```

        if task.status.state == "completed":

```

```

            result_text = task.artifacts.parts.data[0].text

```

```

            return json.loads(result_text) if isinstance(result_text, str) else result_text

```

```

        elif task.status.state in ["failed", "cancelled"]:

```

```

            return None

```

```

        await asyncio.sleep(1.0)

```

```

        task = await agent.get_task(task_id)

```

```

    return None

```

```

    except Exception as e:

```

```

        print(f"A2A request failed for {supplier_url}: {e}")

```

```

    return None

```

```

    async def query_all_suppliers_parallel(self, suppliers: list[dict]) -> dict[str, dict | None]:

```

```

        """Query multiple supplier agents in parallel."""

```

```

        tasks = []

```

```

        self.request.emissions_data = {

```

```

            "supplier_url": s["url"],

```

```

            "material_type": s["material_type"],

```

```

            "quantity_tonnes": s["quantity_tonnes"],

```

```

            "production_period": s["production_period"],

```

```

        ) for s in suppliers

```

```

        results = await asyncio.gather(*tasks, return_exceptions=True)

```

```

        return {

```

```

            s["supplier_name"]: (r if not isinstance(r, Exception) else None)

```

```

            for s, r in zip(suppliers, results)

```

How Judges See A2A Working

During the demo, show this real-time sequence in the Antigravity 2.0 dashboard:

1. **[10:23:38]** Logistics Optimizer Agent: "Missing emission data for PackagingLtd. Initiating A2A discovery."
2. **[10:23:39]** A2A Client → GET
`https://supplier-packaging.ecoflow.app/.well-known/agent.json`
3. **[10:23:39]** A2A Discovery: "PackagingLtd agent found. Capabilities: emissions_verification, certification_lookup"
4. **[10:23:40]** A2A Client → POST /a2a/tasks/send — Task: "Provide CO2e for 120t corrugated cardboard, 2026-Q1"
5. **[10:23:41]** PackagingLtd Agent: "Calculating from internal production database..."
6. **[10:23:42]** A2A Response: {total_embedded_co2e_kg: 84000, factor: 700 kg/t, cert: BV-PKG-2026-019}
7. **[10:23:42]** Carbon Engine Agent: "Supplier data received. Recalculating emission totals."
8. **[10:23:43]** Compliance Auditor Agent: "Re-evaluating compliance. Score updated: 0.72 → 0.89"

7. FastMCP Design

Overview

Three independent FastMCP servers expose the core data infrastructure as tools that agents can discover and call via the Model Context Protocol. Each runs as an independent Cloud Run service.

7.1 carbon-db-mcp

Purpose: Provides emission factors for material production (Scope 3 Category 1) and energy sources from authoritative public databases.

Data Sources:

- EPA Emission Factors for Greenhouse Gas Inventories (April 2024 edition)
- IEA World Energy Outlook 2024 (country-specific grid emission factors)
- World Steel Association Climate Change Report 2024
- Ecoinvent 3.10 sector averages (public subset)
- IPCC AR6 Table 2.A.2 (GWP100 values)

Exposed Tools:

```
Python
# carbon_mcp/server.py
from fastmcp import FastMCP
```

```
import sqlite3
```

```
mcp = FastMCP("carbon-db-mcp" description="EPA and IEA emission factors database")
```

```
@mcp.tool()
```

```
def get_material_emission_factor(material_type: str, production_country: str = "GLOBAL", year: int = 2024) -> dict:
```

```
    """
```

```
    Retrieve emission factor for material production (Scope 3 Category 1).
```

```
    Returns emission factor in kg CO2e per tonne of material produced.
```

```
    """
```

```
    conn = sqlite3.connect("/data/emission_factors.db")
```

```
    # [Implementation queries SQLite database]
```

```
    pass
```

```
@mcp.tool()
```

```
def get_grid_emission_factor(country: str, year: int = 2024) -> dict:
```

```
    """
```

```
    Retrieve electricity grid emission factor for a country.
```

```
    Source: IEA World Energy Outlook 2024.
```

```
    """
```

```
    pass
```

```
@mcp.tool()
```

```
def get_gwp100_factor(gas: str) -> float:
```

```
    """
```

```
    Return IPCC AR6 GWP100 value for a greenhouse gas.
```

```
    Supported: CO2, CH4, N2O, SF6, HFCs
```

```
    """
```

```
    gwp_values = {"CO2": 1.0, "CH4": 27.9, "N2O": 273.0, "SF6": 25200.0}
```

```
    return gwp_values.get(gas, 1.0)
```

```
@mcp.tool()
```

```
def list_supported_materials() -> list[str]:
```

```
    """List all material types with available emission factors."""
```

```
    pass
```

```
@mcp.tool()
```

```
def search_emission_factors(query: str, limit: int = 10) -> list[dict]:
```

```
    """Semantic search across emission factor database."""
```

```
    pass
```

```
if __name__ == "__main__":
```

```
    mcp.run(transport="sse", host="0.0.0.0", port=8081)
```

Sample Data (SQLite emission_factors.db):

```
SQL
CREATE TABLE material_emission_factors (
  id INTEGER PRIMARY KEY
  material_type TEXT NOT NULL
  material_code TEXT
  country TEXT DEFAULT 'GLOBAL'
  year INTEGER DEFAULT 2024
  factor_kg_co2e_per_tonne REAL NOT NULL
  source TEXT NOT NULL
  methodology TEXT
  confidence REAL DEFAULT 0.85
  notes TEXT
)

INSERT INTO material_emission_factors VALUES
1 'hot_rolled_steel' '7208' 'GLOBAL' 2024 1800 'World Steel Association 2024' 'Cradle-to-gate, AR6 GWP100' 0.90 'Average of integrated and EAF routes'
2 'hot_rolled_steel' '7208' 'DE' 2024 1620 'WSA 2024, Destatis energy mix' 'Cradle-to-gate, DE grid 2024' 0.87 'German steel sector, high EAF share'
3 'hot_rolled_steel' '7208' 'CN' 2024 2100 'WSA 2024, IEA China grid' 'Cradle-to-gate, CN grid 2024' 0.85 'China predominantly BF-BOF route'
4 'aluminium_primary' '7601' 'GLOBAL' 2024 11500 'IAI 2024 Global Aluminium Lifecycle' 'Cradle-to-gate including electrolysis' 0.88 NULL
5 'cement_portland' '2523' 'GLOBAL' 2024 820 'GCCA 2024 Getting the Numbers Right' 'Cradle-to-gate, clinker factor 0.72' 0.92 NULL
6 'polyethylene' '3901' 'GLOBAL' 2024 1900 'PlasticsEurope Eco-profiles 2023' 'Cradle-to-gate HDPE' 0.83 NULL
7 'corrugated_cardboard' '4819' 'GLOBAL' 2024 700 'FEFCO Environmental Statement 2024' 'Cradle-to-gate, recycled content 70%' 0.80 NULL
8 'flat_glass' '7005' 'GLOBAL' 2024 870 'Glass for Europe 2024' 'Cradle-to-gate' 0.82 NULL
```

7.2 cbam-reg-mcp

Purpose: Provides current CBAM regulatory data including commodity scope, carbon price references, declaration requirements, and compliance thresholds.

Data Sources:

- EU Regulation (EU) 2023/956 (full text)
- CBAM Implementing Regulation (EU) 2023/1773
- EU ETS carbon price (via ECX API simulation or cached weekly)

Exposed Tools:

Python

@mcp.tool()

def check_cbam_applicability(hs_code: str, origin_country: str) -> dict:

"""Check if a product is subject to CBAM based on HS code."""

pass

@mcp.tool()

def get_cbam_embedded_emissions_threshold(hs_code: str) -> dict:

"""Return the default value for embedded emissions per Annex III."""

pass

@mcp.tool()

def get_current_carbon_price() -> dict:

"""Return current EU ETS carbon price for CBAM levy calculation."""

pass

@mcp.tool()

def calculate_cbam_levy(embedded_co2e_tonnes: float, product_hs_code: str, origin_country: str, carbon_price_eur: float) -> dict:

"""Calculate CBAM financial obligation implementing Article 7 methodology."""

pass

@mcp.tool()

def get_cbam_declaration_requirements(company_country: str) -> dict:

"""Return CBAM declaration deadlines and documentation requirements."""

pass

@mcp.tool()

def lookup_article(regulation: str, article_number: str) -> str:

"""Retrieve specific article text from CBAM regulation for citation."""

pass

7.3 logistics-mcp

Purpose: Provides transport emission factors by mode, fuel type, and vehicle category for Scope 3 Category 4 (upstream transport and distribution) calculations.

Data Sources:

- GLEC Framework v3.0 (Global Logistics Emissions Council)
- ICAO Carbon Emissions Calculator methodology (air freight)
- IMO 4th GHG Study 2020 (sea freight)

Exposed Tools:

```
Python
@mcp.tool()
def get_transport_emission_factor(transport_mode: str, fuel_type: str = "diesel", load_factor: float = 0.7, vehicle_category: str = "default") -> dict:
    """Return transport emission factor in kg CO2e per tonne-km based on GLEC v3.0."""
    pass

@mcp.tool()
def calculate_transport_emissions(transport_mode: str, quantity_tonnes: float, distance_km: float, fuel_type: str = "diesel") -> dict:
    """Calculate total transport emissions for a shipment leg."""
    pass

@mcp.tool()
def compare_modal_alternatives(origin: str, destination: str, quantity_tonnes: float, current_mode: str) -> list[dict]:
    """Compare emission footprint of alternative transport modes."""
    pass
```

8. Data Strategy

What to Use (Real Public Data)

EPA Emission Factors (Primary Source)

- **URL:**
https://www.epa.gov/system/files/documents/2024-02/emission-factors_feb-2024.pdf
- **What to extract:** Table 2.1 (Stationary combustion), Table 3 (Mobile combustion), Table 4 (Fugitive emissions)
- **Materials covered:** Steel, cement, glass, plastics, paper products

CBAM Regulation Text

- **URL:** <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:32023R0956>
- **What to extract:** Annex I (covered products with HS codes), Annex III (default embedded emission values), Article 7 (calculation methodology), Article 22 (certificate pricing)

GLEC Framework Transport Factors

- **URL:**
<https://www.smartfreightcentre.org/en/our-programs/global-logistics-emissions-council/glec-framework/>
- **Key factors:**
 - Road (HGV >32t, diesel): 0.062 kg \$CO_2e\$/tonne-km
 - Rail (freight, EU average): 0.022 kg \$CO_2e\$/tonne-km
 - Sea (container, large vessel): 0.008 kg \$CO_2e\$/tonne-km
 - Air freight: 0.500 kg \$CO_2e\$/tonne-km

World Steel Association Data

- **URL:**
<https://worldsteel.org/steel-topics/sustainability/climate-change-and-the-production-of-iron-and-steel/>
- **Key factors:**
 - Integrated steelmaking (BF-BOF): 2.1 t \$CO_2e\$/t steel
 - Electric arc furnace (EAF): 0.7 t \$CO_2e\$/t steel (EU grid, 2024)
 - Global average: 1.8 t \$CO_2e\$/t steel

Sample Logistics Dataset (For Demo)

Create a realistic CSV named demo_audit_2026_Q1.csv:

```
Code snippet
invoice,supplier_name,supplier_country,material_type,hs_code,quantity_tonnes,transport_mode,distance_km,origin_city,destination_city,invoice_ref
SteelCo,
"IG DE hot rolled steel",720851450,0,rail,340,Hamburg,Rotterdam,INV-2026-0234
SteelCo,
"IG DE cold rolled steel",720915120,0,rail,340,Hamburg,Rotterdam,INV-2026-0234
FreightHub,
"mbH DE logistics service",999999570,0,rail,340,Hamburg,Rotterdam,INV-2026-0234
PackagingLtd,PL,corrugated_cardboard,48191085,0,road,890,Warsaw,Rotterdam,INV-
```

```
INV-2026-024
ChemBase
CAFR polyethylene hdpe,3901,20,62,0,road,520,Lyon,Rotterdam,INV-2026-024
GlassTech BV NL float_glass,7005,10,38,0,road,45,Eindhoven,Rotterdam,INV-2026-025
AlumCo
AS NO aluminium primary,7601,10,22,0,sea,1840,Bergen,Rotterdam,INV-2026-026
AlumCo
AS NO aluminium extrusions,7604,10,8,0,sea,1840,Bergen,Rotterdam,INV-2026-026
```

What to Simulate (Honestly)

- **Supplier agent databases:** The supplier agents run as real services but execute calculations over representative, predefined internal database dictionaries.
- **EU ETS carbon price:** Cache a representative static price (€65/tonne) rather than configuring complex real-time trading APIs.
- **Third-party verification certificates:** Generate realistic mock certificate strings labeled explicitly as "simulated for demonstration."

What to Avoid

- **Synthetic emission factors:** Never invent calculation coefficients. Use valid public baseline numbers.
- **Overstating compliance coverage:** Explicitly document your scope boundaries (e.g., 8 material profiles across 3 active vendor targets).
- **Real company names:** Mitigate corporate risk by leveraging explicit fictional placeholders.

9. Repository Structure

```
Plaintext
ecoflow
├── README.md           # Competition readme — judge-facing
├── ARCHITECTURE.md     # Technical architecture deep dive
├── pyproject.toml      # Project dependencies (uv)
├── docker-compose.yml  # Local development stack
├── env.example         # Environment variable template
├── Makefile            # Common development commands
├──
└── ecoflow_hub/       # Main orchestration service
```



```
└── Dockerfile
└── main.py # FastAPI app + ADK agent initialization
└── agents
    ├── init.py
    ├── orchestrator.py # Root orchestrator agent (ADK LimAgent)
    ├── document_extractor.py # Document parsing agent
    ├── carbon_engine.py # Emission calculation agent
    ├── compliance_auditor.py # CBAM compliance agent
    ├── logistics_optimizer.py # A2A coordinator + optimization agent
    └── a2a
        ├── init.py
        ├── client.py # A2A protocol client
        ├── supplier_registry.py # Supplier agent URL registry
        ├── schemas.py # A2A message schemas (Pydantic)
        └── tools
            ├── init.py
            ├── audit_tools.py # Session init, report generation
            ├── storage_tools.py # GCS, BigQuery integration
            └── report_generator.py # PDF audit report generation
└── sessions
    ├── init.py
    ├── vertex_session.py # VertexAISessionService wrapper
    └── config.py # Configuration management
└──
└── supplier_agents/ # Independently deployable supplier agents
    ├── base
    │   ├── init.py
    │   └── base_supplier.py # Base class for all supplier agents
    ├── schemas.py # Shared Pydantic schemas
    └── steelco
        ├── Dockerfile
        ├── main.py # SteelCo A2A server
        ├── emissions_db.py # SteelCo internal emissions data
        ├── agent_card.json # A2A agent card definition
        └── freighthub
            ├── Dockerfile
            ├── main.py
            ├── transport_db.py
            ├── agent_card.json
            ├── packaging101
            │   ├── Dockerfile
            │   ├── main.py
            │   └── packaging_db.py
```

```
└── agent_card.json
└──
└── mcp_servers/ # FastMCP servers (independently deployable)
└── carbon_db
└── └── Dockerfile
└── └── server.py # FastMCP server
└── └── data
└── └── └── emission_factors.db # Pre-populated SQLite database
└── └── └── scripts
└── └── └── └── populate_db.py # Database population script
└── cbam_reg
└── └── Dockerfile
└── └── server.py
└── └── data
└── └── └── cbam_regulation.db # CBAM regulatory data
└── └── └── carbon_prices.json # Cached EU ETS prices
└── └── └── logistics
└── └── └── Dockerfile
└── └── └── server.py
└── └── └── data
└── └── └── └── transport_factors.db # GLEC Framework factors
└──
└── frontend/ # Streamlit dashboard
└── └── app.py # Main Streamlit application
└── └── pages
└── └── └── 01_carbon_dashboard.py # Main carbon overview
└── └── └── 02_supplier_network.py # Supplier risk matrix
└── └── └── 03_cbam_audit.py # CBAM compliance details
└── └── └── 04_agent_activity.py # Live agent decision log
└── └── └── 05_demo_mode.py # Judge demo with scripted scenarios
└── └── components
└── └── └── charts.py # Plotly chart components
└── └── └── a2a_visualizer.py # A2A communication visualizer
└── └── └── report_viewer.py # Embedded PDF report viewer
└── └── assets
└── └── └── demo_data
└── └── └── └── demo_audit_2026_Q1.csv
└──
└── data/ # Data assets
└── └── emission_factors
└── └── └── epa_2024_factors.csv
└── └── └── glec_transport_v3.csv
└── └── └── wsassoc_steel_2024.csv
```

```
├── regulatory
│   ├── cbam_annex1_hs_codes.csv
│   ├── cbam_annex3_defaults.csv
│   ├── demo
│   └── demo_audit_2026_Q1.csv
├──
├── notebooks/ # Kaggle submission notebooks
│   ├── ecoflow_competition_notebook.ipynb
│   └── architecture_walkthrough.ipynb
├──
├── tests/ # Test suite
│   ├── unit
│   │   ├── test_carbon_calculations.py
│   │   ├── test_cbam_compliance.py
│   │   └── test_document_extraction.py
│   ├── integration
│   │   ├── test_a2a_communication.py
│   │   ├── test_mcp_servers.py
│   │   └── test_full_pipeline.py
│   └── a2a
│       └── test_full_audit_workflow.py
├──
├── deployment/ # Infrastructure and deployment
│   ├── cloudbuild.yaml
│   ├── terraform/ # (Optional) Terraform IaC
│   └── scripts
│       ├── deploy_all.sh # Deploy all services to Cloud Run
│       ├── seed_databases.py # Populate MCP server databases
│       └── run_demo.sh # Start local demo environment
├──
├── docs/ # Documentation
│   ├── architecture
│   │   ├── system_overview.md
│   │   ├── a2a_protocol.md
│   │   └── agent_design.md
│   ├── api
│   │   ├── hub_api.md
│   │   ├── mcp_tools.md
│   │   └── a2a_schema.md
│   ├── demo
│   └── demo_script.md
```

10. Database Design

10.1 Audit Database (BigQuery)

SQL

-- Main audit records table

CREATE TABLE IF NOT EXISTS ecoflow.audits.audit_records

audit_id STRING NOT NULL

company_name STRING NOT NULL

reporting_period STRING NOT NULL

created_at TIMESTAMP NOT NULL

completed_at TIMESTAMP

status STRING

total_scope3_co2e_tonnes FLOAT64

scope3_cat1_co2e_tonnes FLOAT64

scope3_cat4_co2e_tonnes FLOAT64

cbam_compliance_score FLOAT64

cbam_estimated_levy_eur FLOAT64

sec_disclosure_required BOOL

supplier_count INT64

a2a_requests_made INT64

a2a_requests_successful INT64

report_url STRING

PRIMARY KEY (audit_id) NOT ENFORCED

;

-- Supplier-level emissions breakdown

CREATE TABLE IF NOT EXISTS ecoflow.audits.supplier_emissions

audit_id STRING NOT NULL

supplier_name STRING NOT NULL

supplier_country STRING

material_type STRING

quantity_tonnes FLOAT64

material_co2e_tonnes FLOAT64

transport_co2e_tonnes FLOAT64

total_co2e_tonnes FLOAT64

emission_factor_source STRING

```

data_quality STRING
cbam_applicable BOOLEAN
cbam_levy_eur FLOAT64
]

-- Agent decision log for observability
CREATE TABLE IF NOT EXISTS ecoflow.audits.agent_decisions
(
  audit_id STRING NOT NULL
  timestamp TIMESTAMP NOT NULL
  agent_name STRING NOT NULL
  action STRING NOT NULL
  input_summary STRING
  output_summary STRING
  tool_calls JSON
  a2a_calls JSON
  duration_ms INT64
)

```

10.2 Emission Factors Database (SQLite — carbon-db-mcp)

```

SQL
CREATE TABLE material_emission_factors
(
  id INTEGER PRIMARY KEY AUTOINCREMENT
  material_type TEXT NOT NULL
  material_name TEXT
  hs_code TEXT
  country TEXT DEFAULT 'GLOBAL'
  year INTEGER NOT NULL
  factor_kg_co2e_per_tonne REAL NOT NULL
  lower_bound REAL
  upper_bound REAL
  source TEXT NOT NULL
  source_url TEXT
  methodology TEXT
  scope TEXT DEFAULT 'cradle_to_gate'
  are_dwp100_applied INTEGER DEFAULT 1
)

```

```

confidence REAL DEFAULT 0.85
notes TEXT
created_at TEXT DEFAULT CURRENT_TIMESTAMP

```

```

CREATE INDEX idx_material_country ON material_emission_factors(material_type, country);
CREATE INDEX idx_hs_code ON material_emission_factors(hs_code);

```

```

CREATE TABLE grid_emission_factors (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  country TEXT NOT NULL,
  year INTEGER NOT NULL,
  factor_kg_co2e_per_kwh REAL NOT NULL,
  source TEXT NOT NULL,
  renewable_share REAL,
  notes TEXT
)

```

```

CREATE TABLE gwp100_values (
  gas TEXT PRIMARY KEY,
  gwp100 REAL NOT NULL,
  ipcc_report TEXT DEFAULT 'AR6',
  notes TEXT
)

```

10.3 CBAM Regulatory Database (SQLite — cbam-reg-mcp)

```

SQL
CREATE TABLE cbam_covered_products (
  hs_code TEXT PRIMARY KEY,
  product_description TEXT NOT NULL,
  cbam_sector TEXT,
  annex_reference TEXT,
  in_force_date TEXT DEFAULT '2026-01-01',
  notes TEXT
)

```

```
CREATE TABLE cham_default_values
(
  id INTEGER PRIMARY KEY AUTOINCREMENT
  hs_code TEXT NOT NULL
  product_description TEXT NOT NULL
  default_embedded_co2e_tonne_per_tonne REAL NOT NULL
  reference_country TEXT DEFAULT 'GLOBAL'
  regulation_reference TEXT
  valid_from TEXT
  valid_to TEXT
)
```

```
CREATE TABLE carbon_price_history
(
  id INTEGER PRIMARY KEY AUTOINCREMENT
  price_eur_per_tonne REAL NOT NULL
  date TEXT NOT NULL
  source TEXT DEFAULT 'EU_ETS_cached'
  notes TEXT
)
```

```
CREATE TABLE regulatory_articles
(
  id INTEGER PRIMARY KEY AUTOINCREMENT
  regulation TEXT NOT NULL
  article_number TEXT NOT NULL
  article_title TEXT
  full_text TEXT
  summary TEXT
)
```

11. API Design

11.1 EcoFlow Hub REST API

```
Plaintext
POST /api/v1/audit
Content-Type: multipart/form-data

```

"company_name": "Acme Manufacturing GmbH"

"reporting_period": "2026-Q1"

"target_market": "EU"

"document": "<file>

|

"response_202": "Accepted"

|

"audit_id": "ECO-2026-Q1-00042"

"status": "RUNNING"

"estimated_duration_seconds": 45

"status_url": "/api/v1/audits/ECO-2026-Q1-00042/status"

"websocket_url": "wss://ecoflow-hub.run.app/ws/audits/ECO-2026-Q1-00042"

|

Plaintext

GET /api/v1/audits/{audit_id}

Response 200 OK

|

"audit_id": "ECO-2026-Q1-00042"

"status": "COMPLETED"

"company_name": "Acme Manufacturing GmbH"

"total_scope3_co2e_tonnes": 12473

"cbam_compliance_score": 0.89

"cbam_estimated_levy_eur": 49892

"report_url": "https://storage.googleapis.com/ecoflow-reports/ECO-2026-Q1-00042.pdf"

"supplier_summary": [...]

"agent_activity": [...]

|

Plaintext


```
GET /api/v1/audits/{audit_id}/status
WebSocket: wss://ecoflow-hub.run.app/ws/audits/{audit_id}
stream of agent events
{"event": "agent_action", "agent": "document_extractor", "action": "parsing.pdf",
"timestamp": "..."}
{"event": "a2a_request", "target": "supplier-steelco", "status": "sent", "timestamp": "..."}
{"event": "a2a_response", "source": "supplier-steelco", "status": "received", "timestamp": "..."}
{"event": "compliance_result", "score": 0.89, "status": "compliant with warnings",
"timestamp": "..."}
{"event": "audit_complete", "report_url": "...", "timestamp": "..."}

```

Plaintext

```
POST /api/v1/audits/{audit_id}/human-review
Content-Type: application/json
{
  "reviewer": "compliance_officer@acme.com",
  "action": "approve" | "reject" | "request_revision",
  "notes": "...",
}
```

11.2 Supplier Agent A2A API

Plaintext

```
GET /well-known/agent.json → AgentCard (see Section 6)
POST /a2a/tasks/send → Task creation (see Section 6)
GET /a2a/tasks/{task_id} → Task status and result
GET /health → {"status": "healthy", "agent": "supplier-steelco", "version": "1.0.0"}

```

11.3 FastMCP Tool API

MCP servers expose tools via the SSE transport. Agents connect using:

```
Python
```

```
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset, SseServerParams
```

```
toolset = MCPToolset()
```

```
connection_params=SseServerParams
```

```
url="https://carbon-db-mcp.ecoflow.app/sse"
```

```
headers= "Authorization" f"Bearer {API_KEY}"
```

12. Frontend Design

Dashboard Architecture

The frontend is a Streamlit multi-page application with 5 views, deployed alongside the hub on Cloud Run.

Page 1: Carbon Intelligence Overview

- **Top Metrics Row:** Total Scope 3 CO₂e (1,247.3 tonnes), CBAM Score (89%), Estimated Levy (€49,892), Supplier Verification Rate (7/8 verified).
- **Supplier Emissions Treemap:** Plotly treemap showing each supplier as a block, sized by \$CO₂e\$ contribution. Color represents verification status.
- **Scope 3 Category Breakdown:** Horizontal bar chart mapping Category 1 (Purchased Goods) vs Category 4 (Upstream Transport).

Page 2: Supplier Network View

- **A2A Communication Visualizer:** Real-time network topology graph highlighting active A2A execution lines between the hub and supplier instances.
- **Supplier Risk Matrix:** Scatter plot analyzing emission footprint volumes against supplier primary data fidelity scales.

Page 3: CBAM Audit View

- **CBAM Levy Breakdown:** Waterfall chart showing total embedded carbon down to calculated net import duty structures.
- **Compliance Checklist:** Interactive audit grid matching parsed line items against statutory Annex criteria.

Page 4: Agent Activity View

Plaintext

```
10:23:38 🔍 Document Extractor: Parsed 8 line items from shipping manifest
10:23:39 🤖 Carbon Engine: Queried carbon-db-mcp for hot-rolled-steel/DE → 1,620 kg CO2e
10:23:40 🤖 Logistics Optimizer: PackagingLtd data missing — initiating A2A request
10:23:42 🤝 A2A — PackagingLtd: Agent discovered. Skill: emissions_verification
10:23:43 🤝 A2A — PackagingLtd: Total co2e: kg: 59500, factor: 700 kg/t, cert: EV-PKG-2026-015
10:23:44 ⚖️ Compliance Auditor: CBAM score: 0.72 → 0.89 after A2A verification
10:23:45 🎯 Orchestrator: Audit complete. Generating PDF report.
```

Page 5: Demo Mode

Pre-configured execution interface triggering the demo_audit_2026_Q1.csv mock batch pipeline instantly to allow seamless evaluation for competition judges without file prerequisites.

13. AI-Assisted Development Workflow

Cursor / Claude Code Prompts

Architecture Setup Prompt

Plaintext

```
Create a Python project for EcoFlow, a federated supply chain carbon auditing system using Google ADK 2.0 and A2A protocol.
```

Project structure:

```
ecoflow/ hub/ — main orchestration service (FastAPI + ADK 2.0 agents)
supplier_agents/ — independently deployable A2A agents (steelco, freighthub, packagingltd)
mcp_servers/ — FastMCP servers (carbon_db, cham_reg, logistics)
frontend/ — Streamlit dashboard
tests/ — pytest test suite
```

```
Requirements
- Python 3.11+
- google-adk>=2.0.0
- a2a-sdk>=1.0.0
- fastmcp>=0.5.0
- fastapi>=0.111.0
- streamlit>=1.35.0
- google-generativeai>=0.7.0
- google-cloud-aiplatform>=1.60.0
- reportlab>=4.2.0
- plotly>=5.22.0
- sqlite3 (stdlib)
- google-cloud-bigquery>=3.25.0
- google-cloud-storage>=2.17.0
- pydantic>=2.7.0
```

```
Create pyproject.toml with all dependencies, a Makefile with common commands (make
install, make dev, make test, make deploy), and docker-compose.yml with all 7 services
Include .env.example with all required environment variables
```

ADK 2.0 Orchestrator Agent Prompt

Plaintext

```
Create the EcoFlow Orchestrator Agent using Google ADK 2.0
File: ecoflow_hub/agents/orchestrator.py
```

```
Use
- google.adk.agents.LlmAgent as the root agent
- google.adk.agents.SequentialAgent for the main pipeline
- VertexAISessionService from google.adk.sessions for state management
- model="gemini-2.5-pro"
```

```
The orchestrator coordinates 4 sub-agents in sequence:
1. DocumentExtractorAgent — parses logistics documents
2. CarbonEngineAgent — calculates emissions using MCP tools
3. ComplianceAuditorAgent — checks CBAM compliance
```

4. LogisticsOptimizerAgent — initiates A2A requests and optimizes

include

- Session initialization with audit_id generation
- Error handling with retry logic (max 3 attempts per sub-agent)
- State passing between agents using session context
- Tool: generate_audit_report() that creates PDF using ReportLab
- Tool: store_result_bigquery() that saves audit to BigQuery
- Complete system prompt focused on CBAM compliance auditing

The orchestrator must return a typed AuditResult Pydantic model. Include all imports. Use async throughout. No TODO comments

A2A Supplier Agent Prompt

Plaintext

Create a complete A2A supplier agent for SteelCo AG using the a2a-sdk Python package
File: supplier_agents/steelco/main.py

Requirements

1. Implement AgentCard at GET /well-known/agent.json with
 - id: "supplier-steelco-agent"
 - skill: "emissions_verification"
 - URL: read from AGENT_URL environment variable
2. Implement A2AStarlettApplication with
 - AgentExecutor subclass: SteelCoExecutor
 - execute() method that parses incoming text, matches against an internal sqlite or dict database, and returns structured carbon metrics
3. Use Gemini 2.5 Flash to parse natural language task descriptions to structured parameters
4. Include FastAPI health endpoint at GET /health

Include Dockerfile. Use uvicorn as the ASGI server. All async. Complete imports. Production-ready error handling.

FastMCP Carbon Database Server Prompt

Plaintext

Create a FastMCP server for carbon emission factors using the fastmcp Python package.
File: mcp_servers/carbon_db/server.py

Implement these tools with complete docstrings:

1. get_material_emission_factor(material_type: str, production_country: str = "GLOBAL", year: int = 2024) -> dict
2. get_grid_emission_factor(country: str, year: int = 2024) -> dict
3. get_gwp100_factor(gas: str) -> float
4. list_supported_materials() -> list[str]
5. search_emission_factors(query: str, limit: int = 10) -> list[dict]

Also create: mcp_servers/carbon_db/scripts/populate_db.py to populate a local SQLite database with real baseline material emissions mapping data from public EPA datasets.
Run as SSE server on port 8081. Include Dockerfile.

Streamlit Dashboard Prompt

Plaintext

Create a Streamlit multi-page dashboard for EcoFlow carbon auditing system.
File: frontend/app.py (main entry) + pages/ directory

Page 1 - Carbon Overview (pages/01_carbon_dashboard.py)

- 4 metric cards: Total CO2e, CBAM Score, CBAM Levy, Verification Rate
- Plotly treemap of emissions by supplier
- Horizontal stacked bar chart: Scope 3 Cat 1 vs Cat 4 by supplier

Page 4 - Agent Activity (pages/04_agent_activity.py)

- Auto-scrolling log of agent decisions fetched via WebSocket connection to hub
- Color-coded cleanly by agent boundaries

Page 5 - Demo Mode (pages/05_demo_mode.py)

One-click demo initialization triggering execution over preloaded CSV files to simulate full analytical agent tracking streams cleanly

Test Generation Prompt

Plaintext

generate a comprehensive pytest test suite for EcoFlow

file: tests/unit/test_carbon_calculations.py

test the Carbon Engine's calculation logic covering purchased goods emissions math, transport category coefficients, and outlier variance anomaly detection flags

file: tests/integration/test_a2a_communication.py

use httpx AsyncClient to validate network communication schemas against deployed mock local A2A worker microservice targets

Deployment Prompt

Plaintext

create deployment scripts for EcoFlow to Google Cloud Run

file: deployment/scripts/deploy_all.sh

Write a clean script building and pushing all 7 primary microservice container layers onto GCP artifact repositories, running standard target provisioning updates seamlessly under cloud Run architectures, include a basic endpoint connectivity loop verification health checker wrapper block

14. Day-by-Day Execution Plan

Day 1: Foundation & Core Pipeline

- **Goal:** Establish baseline local project workspace and initialize primary parsing and calculation logic modules via local MCP components directly.
- **Timeline Split:**
 - **H1-H2:** Workspace setup, package validation, and directory tree instantiation.
 - **H3-H4:** FastMCP baseline data server setup and reference DB initialization.
 - **H5-H6:** Document Extraction parsing prompt validation and performance checking.
 - **H7-H8:** Initial local Carbon execution graph pipelines assembly and evaluation.

Day 2: A2A Protocol & Supplier Agents

- **Goal:** Initialize distinct microservice mock instances for separate suppliers and configure core programmatic network exchange configurations via the A2A SDK primitives.
- **Timeline Split:**
 - **H1-H2:** SteelCo worker image baseline creation and local validation loops.
 - **H3-H4:** Central Hub configuration hooks integration for automated remote Discovery.
 - **H5-H6:** Replicating standalone parallel client workers over distinct service ports.
 - **H7-H8:** Execution routing validation loops over complex missing context pathways.

Day 3: Compliance Auditor, MCP Servers & CBAM

- **Goal:** Incorporate dedicated statutory calculation parameters alongside targeted loop workflows for autonomous remediation processing structures.
- **Timeline Split:**
 - **H1-H2:** Setting up standalone regulatory validation tools via dedicated FastMCP tasks.
 - **H3-H5:** Incorporating legislative citation parameters into standard verification states.
 - **H6-H7:** Designing conditional workflow loops utilizing native ADK architecture paths.
 - **H8:** Full data flow pipeline functional validation under composite mock data scenarios.

Day 4: Deployment & Frontend

- **Goal:** Transition local architecture frameworks into live GCP Cloud Run execution points linked directly with user visual control interfaces.
- **Timeline Split:**
 - **H1-H2:** Multi-container docker-compose assembly and environmental mapping checks.
 - **H3-H5:** GCP Cloud Run service stack cluster orchestration deployments.
 - **H6-H7:** Activating multi-page Streamlit views linked with central service sockets.
 - **H8:** Activating Antigravity dashboard pipeline configuration hooks.

Day 5: Polish, Testing & Submission

- **Goal:** Final target artifact generation, diagnostic evaluation execution, and validation writeup aggregation for platform packaging.
- **Timeline Split:**
 - **H1-H2:** Assembling ReportLab programmatic PDF layout generator functions.
 - **H3-H4:** Finalizing full integration testing scripts and error management validation hooks.
 - **H5-H7:** Curating visual assets, system logs, and data summary charts for the Kaggle interface.
 - **H8:** Executing clean project deployment submission verification checkpoints.

15. Testing Strategy

Unit Tests

Python

```
# tests/unit/test_carbon_calculations.py
```

```
import pytest
```

```
from ecoflow_hub.agents.carbon_engine import CarbonEngineCalculator
```

```
class TestScope3Calculations:
```

```
    def test_category1_basic(self):
```

```
        calc = CarbonEngineCalculator()
```

```
        result = calc.calculate_scope3_cat1()
```

```
        material_type = "hot_rolled_steel"
```

```
        quantity_tonnes = 450.0
```

```
        emission_factor_kg_per_tonne = 1800.0
```

```
        factor_source = "WSA 2024"
```

```
        assert result.co2e_tonnes == pytest.approx(810.0, rel=1e-3)
```

```
        assert "WSA 2024" in result.calculation_audit
```

```
        assert result.category == "scope3_cat1_purchased_goods"
```

```
    def test_category4_rail(self):
```

```
        calc = CarbonEngineCalculator()
```

```
        result = calc.calculate_scope3_cat4()
```

```
        quantity_tonnes = 570.0
```

```
        distance_km = 340.0
```

```
        transport_mode = "rail"
```

```
        emission_factor_kg_per_tonne_km = 0.022
```

```
        assert result.co2e_tonnes == pytest.approx(4.264, rel=1e-2)
```

A2A Integration Tests

Python

```
# tests/integration/test_a2a_communication.py
```

```
import pytest
```

```
import httpx
```

```
import asyncio
```

```
STEELCO_URL = "http://localhost:8084"
```

```
@pytest.mark.asyncio
```

```
async def test_agent_card_discovery():
```

```
    async with httpx.AsyncClient() as client:
```

```
        response = await client.get(f"{STEELCO_URL}/well-known/agent.json")
```

```
        assert response.status_code == 200
```

```
        card = response.json()
```

```
        assert card["id"] == "supplier-steelco-agent"
```

```
        assert any(["id" == "emissions_verification" for _ in card["skills"]])
```

End-to-End Test

Python

```
# tests/e2e/test_full_audit_workflow.py
```

```
import pytest
```

```
import httpx
```

```
import time
```

```
HUB_URL = "http://localhost:8080"
```

```
def test_full_audit_workflow():
```

```
    with open("data/demo/demo_audit_2026_Q1.csv", "rb") as f:
```

```
        response = httpx.post
```

```
            f"{HUB_URL}/api/v1/audits"
```

```
            data={"company_name": "Test Corp", "reporting_period": "2026-Q1", "target_market": "EU"
```

```
                files={"document": f}}
```

```

        timeout=120.0

    assert response.status_code == 202
    audit_id = response.json()["audit_id"]

    for i in range(90):
        status_resp = httpx.get(f"{HUB_URL}/api/v1/audits/{audit_id}")
        status = status_resp.json()["status"]
        if status == "COMPLETED":
            break
        time.sleep(1.0)

    result = status_resp.json()
    assert result["total_scope3_co2e_tonnes"] == 0
    assert result["report_url"].startswith("https://storage.googleapis.com")

```

16. Demo Strategy

Demo Narrative

- **The Hook (30s):** Every time a product crosses an EU border in 2026, the importer must declare its carbon content under CBAM or face severe penalties. EcoFlow replaces months of manual consulting with a distributed network of AI agents that negotiate carbon metrics directly in seconds.
- **The Problem (30s):** Acme Manufacturing imports complex metal components from multiple regional vendors. They possess nothing but a fragmented, unverified logistics transport invoice sheet.
- **The Execution (3m):** Demonstrate the interface importing files live, showcasing the parsing sequences via active visual tracking timelines, demonstrating the automated A2A supplier query step, and outputting a certified regulatory submission report package.
- **The Close (30s):** EcoFlow demonstrates the true capability of modern enterprise automation protocols by synthesizing distributed network checks directly within production standard benchmarks.

Demo Fallback Plan

- **Network Interruption:** Maintain complete operational configuration variables enabling instant redirect flags to local deployment endpoints via pre-configured local docker targets directly.
- **API Rate Limitations:** Store pre-computed audit results inside localized asset structures allowing interactive dashboard rendering flows to operate deterministically under fallback conditions.

17. Kaggle Submission Strategy

Notebook Structure

- **EcoFlow: Federated Supply Chain Decarbonization & CBAM Auditor**
- **Executive Architecture Graph Summary:** Clear, descriptive positioning statement detailing design metrics.
- **Core Legislative Context Profile:** Detailed structural overview mapping framework properties directly to modern statutory frameworks.
- **Complete System Blueprint Code Blocks:** Ordered sequence of implementation modules from intake parsers up to orchestrator loops.
- **observability Log Output Dumps:** Clean evidence arrays capturing real network tracking communication steps directly.
- **Live Deployment References:** Verifiable live link targets pointing towards working Cloud Run instances directly.

GIF Strategy

Incorporate clear, highly targeted, lightweight screen capture animations showcasing:

1. **Agent Interaction Sequences:** Highlighting log streams rendering autonomously across agent task steps.
2. **Antigravity Visual Overviews:** Capturing cross-node messaging metrics during active supplier handshake executions.
3. **Fidelity Recalibration Adjustments:** Showcasing system gauges shifting dynamically upon processing granular structural metrics.

18. Documentation Plan

README.md (Competition-Facing)

Markdown

EcoFlow 🌿 — Federated Supply Chain Decarbonization & CBAM Auditor

An autonomous multi-agent AI system that audits supply chain carbon footprints and ensures CBAM compliance via federated protocols.

Quick Start

```
```bash
pip install -r requirements.txt
cp .env.example .env
docker-compose up -d
```

Navigate to <http://localhost:8501> to view the Streamlit interface.

```
ARCHITECTURE.md
Detailed technical manual covering microservice topology models, database structures,
security protocols, and comprehensive calculation logic breakdowns.

19. Risk Analysis

Technical Risks

| Risk | Probability | Impact | Mitigation |
|-----|-----|-----|-----|
| **ADK 2.0 API Variance** | Medium | High | Pin deployment manifests to precise release
versions early on Day 1. |
| **AZA Connection Dropouts** | Medium | High | Construct precise local procedural HTTP
fallback channels natively. |
| **Model Context Pipeline Timeouts** | Low | Medium | Configure explicit token response
sounds across heavy lookup calls. |

Competition Risks

| Risk | Probability | Impact | Mitigation |
|-----|-----|-----|-----|
| **Judges Unfamiliar with Sector** | Medium | Medium | Anchor introductory views with
high-impact economic overview summaries. |
| **Platform Integration Failures** | Low | High | Maintain production-ready static database
asset maps within the notebook. |

20. Winning Enhancement

Must-Have Metrics (Baseline, Target Block)
Comprehensive multi-agent architecture utilizing native ADK workflows.
Real, verified cross-node network validation configurations over active services.
Authoritative regulatory references embedded across output data structures.
```

## ## Should-Have Metrics (High Distinction Markers)

- Parallel query processing utilizing non-blocking execution streams
- Automated data self-correction pipelines checking calculation certainty scores
- Live analytical metric visualization charts tracking operational supply logs

## ##

## # 21. Final Winning Roadmap

### ## Minimum Viable Submission (Score: ~60/100)

- Single Kaggle notebook file compiling manual string concatenation checks against simple data sets. No live servers, no true distributed architecture hooks

### ## Strong Submission (Score: ~72/100)

- Multi-agent execution logic loops running inside local work environments backed by visual metric dashboards

### ## Finalist Submission (Score: ~88/100)

- Active cloud-hosted cluster running live distributed data checking loops over independent service endpoints backed by explicit automated log tracing metrics

### ## Likely Winner Submission (Score: ~95/100)

- Full-scale production-ready compliance topology showcasing multi-node negotiation executions using real legislative factors, self-correcting calculation pipelines, human validation intercept modes, and high-fidelity auditable output files

## ##

Document version 1.0 — EcoFlow Competition Blueprint

Kaggle x Google 5-Day Gen AI Agent Championship | June 2026